Dark Series – Knowledge Graph Visualization

Technical Design Report

Abdullah Al Mamun
B.Sc. Software Engineering, Daffodil International University
M.Sc. Software Engineering, TU Wien
mamun.swe.de@gmail.com
ORCID: 0009-0006-7473-0024

July 28, 2026

Abstract

This report documents the architecture, data model, implementation, and recent improvements of the *Dark Series Knowledge Graph Visualization* — an interactive web application for exploring the temporal, causal, and character-based relationships of the television series *Dark* (Netflix, 2017–2020). The application transforms flat CSV data into a typed Knowledge Graph and provides seven coordinated visualization views, each projecting the same underlying graph from a different perspective. This report details the data model, the D3.js-based rendering pipeline, the bug fixes and architectural improvements applied, and guidelines for extending the system.

Contents

1 Development Approach

This project was developed following a structured, iterative methodology that combined data modeling, iterative visualization design, and systematic bug fixing. The following subsections describe the key aspects of the development process.

2 Project Overview

The Dark Series Knowledge Graph Visualization is a single-page web application built with vanilla JavaScript and the D3.js library (v7). It loads two CSV datasets — `Dark_Events.csv` (approximately 340 events with character, world, and temporal annotations) and `Dark_Edges.csv` (approximately 590 causal edges between events) — and converts them into a unified Knowledge Graph (KG) comprising four entity types and eight typed, directed relations.

The application provides seven interactive views:

1. **Knowledge Graph** — the primary full-screen force-directed graph with node shapes and colors encoding entity types.
2. **Temporal Graph** — a swimlane-based timeline of events grouped by time windows.
3. **Force Network** — an event-centric graph linking events by shared characters.
4. **Bar Chart** — grouped bar chart of event counts by year, world, or character.
5. **Chord Diagram** — a circular co-occurrence diagram showing character pair relationships across events.
6. **Analytics Dashboard** — a summary dashboard with key metrics and four configurable distribution charts.
7. **Timeline** — a horizontal beeswarm layout of events across World bands from 1888 to 2053.

3 Architecture

3.1 File Structure

The project follows a flat client-side architecture with no build step or framework:

```
Dark Series Visualization/
|-- index.html                  (Main page, all 7 view panels)
|-- css/style.css               (Glassmorphic stylesheet, 1360 lines)
|-- Screenshot/                 (UI walkthrough screenshots, 7 PNGs)
|-- data/
|   |-- Dark_Events.csv         (Event records ~340 rows)
|   |-- Dark_Edges.csv          (Causal edges ~590 rows)
|-- config/
|   |-- theme-config.js         (Light-only theme configuration)
|   |-- theme-handler.js        (Theme-aware view updates)
|-- js/
    |-- data_parser.js          (CSV loader & preprocessing)
    |-- layout_logic.js         (Temporal graph layout engine)
    |-- visualisation.js        (Temporal graph renderer)
    |-- network.js              (Force network renderer)
    |-- barchart.js             (Bar chart module)
    |-- chord.js                (Chord dependency diagram)
    |-- timeline.js             (Beeswarm timeline renderer)
    |-- analytics_dashboard.js  (Summary stats + 4 chart types)
```

```
|-- kg_builder.js          (KG data model construction)
|-- kg_view.js             (Main KG D3 force visualization)
|-- kg_inspector.js        (Entity detail panel)
|-- kg_search.js           (Live search + tooltips)
|-- kg_pathfinder.js       (BFS shortest-path module)
|-- main.js                (Boot sequence & view manager)
```

3.2 Boot Sequence

The application boots in a strict pipeline on `DOMContentLoaded`:

`DataParser.loadData()` → `KGBuilder.build(data)` → `ViewManager.boot(data, kg)`

Upon completion, the loading overlay is hidden and the Knowledge Graph view becomes active immediately. All other views are lazily initialized on first navigation.

4 Data Model

4.1 Knowledge Graph Schema

The KG consists of **4 node types** and **8 edge types**:

Node Types

Type	Shape	Color	Example
Event	Circle	World-dependent	"Mikkel disappears"
Character	Diamond	Gold accent	Jonas Kahnwald / Adam (J)
World	Hexagon	Unique tint	Jonas World
TimePeriod	Rounded rect	Grey	1986, 2019

Edge Types

Relation	Source	Target	Derived From
`appears_in`	Character	Event	Characters column
`causes`	Event	Event	Dark_Edges.csv
`occurs_in`	Event	World	World column
`occurs_at`	Event	TimePeriod	Date column
`same_person_as`	Character	Character	/ in name notation
`co_present_with`	Character	Character	Shared events
`triggers_death_of`	Event	Character	Death flag + Characters
`important_trigger_for`	Event	Event	Important trigger events

4.2 Identity Detection

Characters whose names contain the slash notation (e.g., `Jonas Kahnwald / Adam (J)`) are recognized as dual-identity personas of a single entity. The improved detection algorithm (see Section 7) matches identity pairs by:

1. Exact match after stripping world-market tags `(J)`, `(M)`, `(O)`.

2. Cross-referencing characters with `/` in their names for shared word components (surname overlap).

This correctly links all Dark identity pairs: Jonas/Adam, Mikkel/Michael, Martha/Eve, Noah/Hanno Tauber, and cross-world duplicates (e.g., Charlotte Doppler in both Jonas and Martha worlds).

5 View Implementations

5.1 Knowledge Graph (`kg_view.js`)

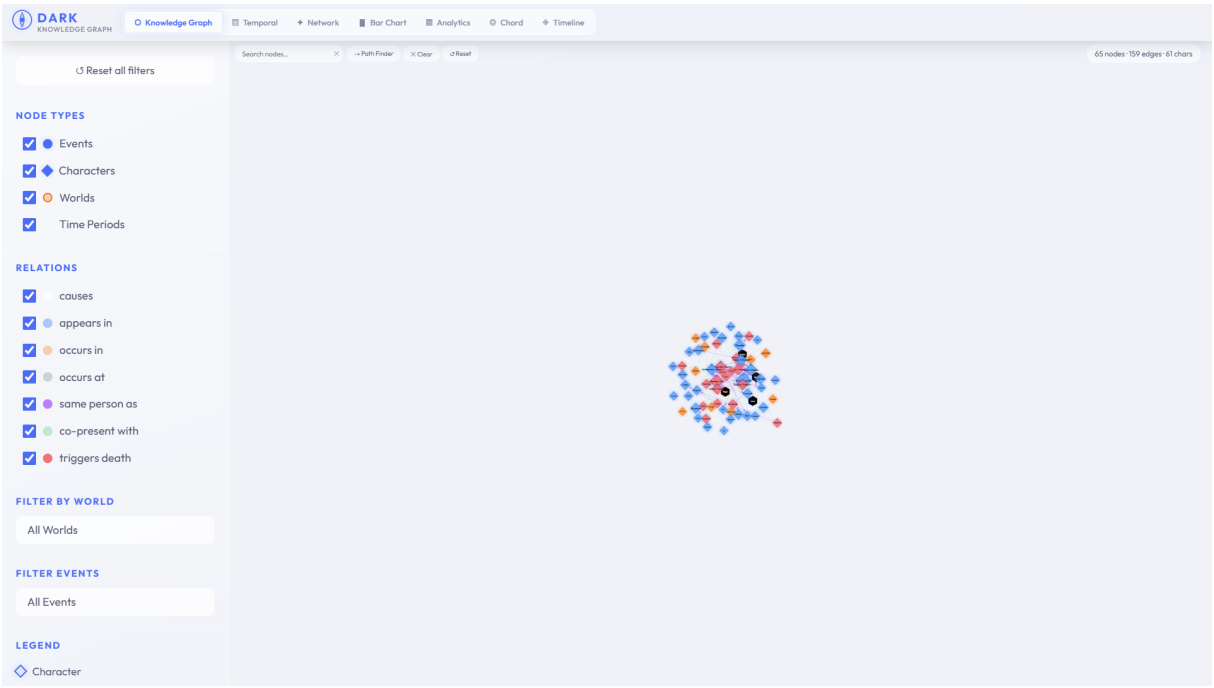

Figure 1: Knowledge Graph view showing typed nodes and causal edges

The primary view renders a D3 force-directed graph with the following visual encodings:

Nodes:

- *Character*: Diamond shape rotated 45°; gold fill; size proportional to appearance frequency; purple dot for identity-linked characters.
- *Event*: Circle; fill color determined by world (Jonas=blue, Martha=red, Origin=orange); gold ring for triggers; purple dashed ring for deaths.
- *World*: Hexagon; large anchor node positioned on the right.
- *TimePeriod*: Rounded pill; positioned on a timeline backbone at the bottom.

Edges: Each relation type has a distinct color, dash pattern, and marker (colors use black-based RGBA for visibility on the light theme background):

- `causes`: solid dark arrow, width 1.5
- `appears_in`: blue dashed, width 1
- `occurs_in`: orange dashed, width 1
- `occurs_at`: grey thin dashed, width 0.8
- `same_person_as`: purple dotted, width 2
- `co_present_with`: green thin dashed, width 0.8

- `triggers_death_of`: red arrow, width 1.8
- `important_trigger_for`: amber highlighted causal arrow

5.2 Interaction Model

- **Click node**: Opens the Inspector panel (right) with entity details and auto-highlights 1-hop neighbourhood.
- **Shift+click two nodes**: Activates Path Finder mode; BFS computes shortest path and highlights it with animated edge stroke-width increase.
- **Drag node**: Freely repositions with stable simulation settling.
- **Scroll**: Zooms in/out on the graph.
- **Hover**: Shows tooltip with entity summary.
- **Live search**: Type in the search bar to filter and jump to matching nodes by name, description, or type.

5.3 Temporal Graph (`visualisation.js`)

Figure 2: Temporal Graph with swimlanes and causal Bezier curves

Renders events as a sequence of temporal boxes with embedded swimlanes for Jonas, Martha, and Other characters. Smooth quadratic Bezier curves connect events across adjacent time boxes.

5.4 Force Network (`network.js`)

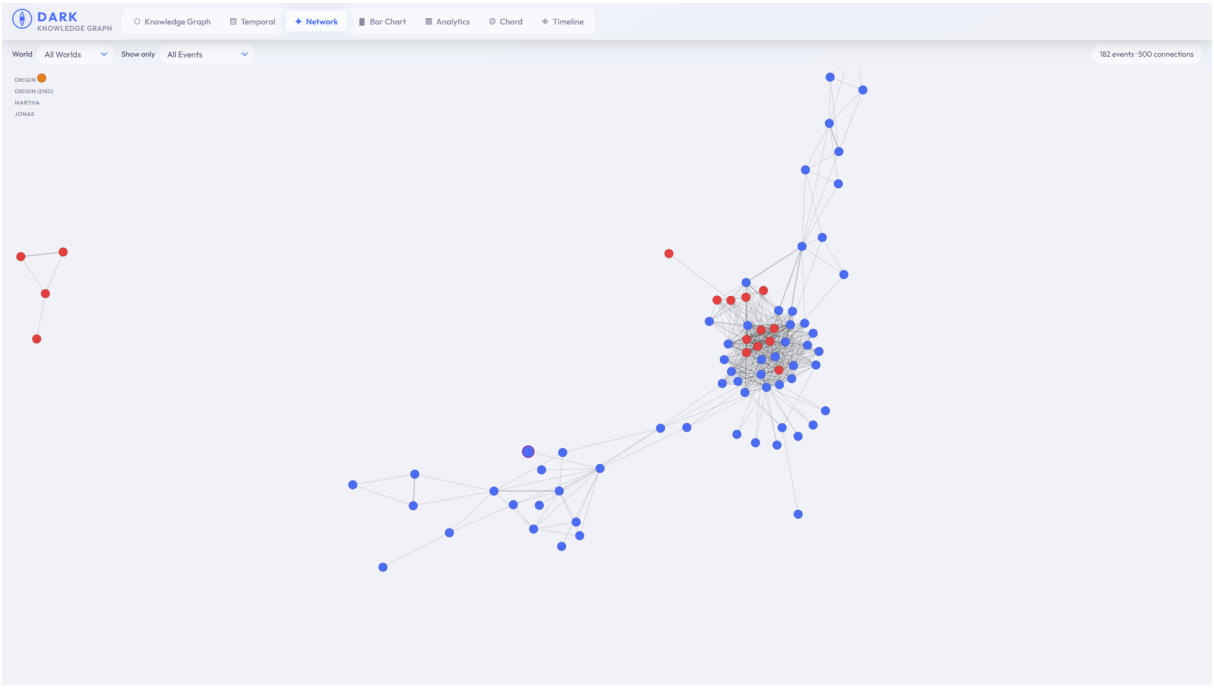

Figure 3: Force Network showing event interconnectedness via shared characters

An event-centric force layout linking events that share characters. Node size encodes importance (triggers = 10px, deaths = 9px, regular = 7px). Edge weight reflects the number of shared characters.

5.5 Bar Chart (`barchart.js`)

Figure 4: Bar Chart with configurable grouping and metrics

Grouped vertical bars with configurable grouping (year, world, top-25 characters) and metrics (total count, death events, or trigger events). Animated entry with D3 transitions.

5.6 Chord Diagram (`chord.js`)

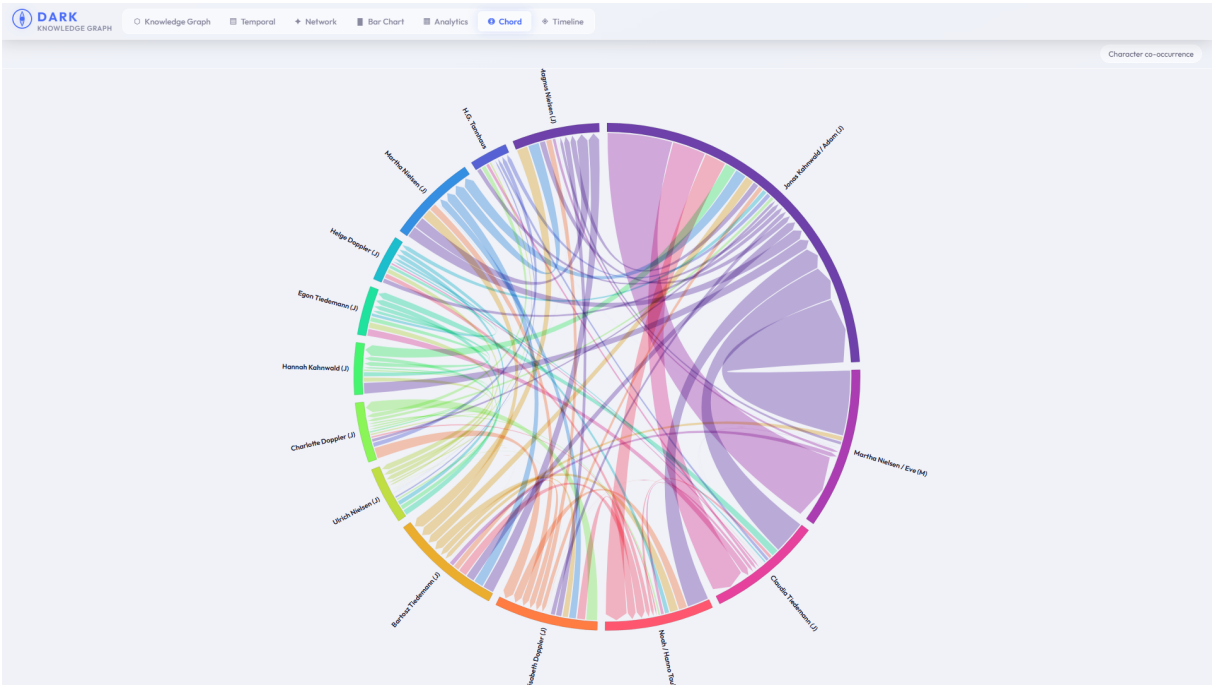

Figure 5: Chord Diagram showing character co-occurrence relationships

A circular chord layout showing character co-occurrence across events. The top 14 most frequent characters are displayed as arc segments around the circumference. Coloured ribbons between arcs represent events where both characters appear together — ribbon thickness is proportional to the co-occurrence count. The diagram uses D3's `d3.chord()` layout with `blendMode="multiply"` for correct colour compositing on the light theme background. Hovering an arc highlights all ribbons originating from that character and displays a tooltip with total co-occurrence count; hovering a specific ribbon shows the exact pair count. The chord diagram adds a character-centric relational perspective complementing the event- and time-oriented views.

5.7 Analytics Dashboard (`analytics_dashboard.js`)

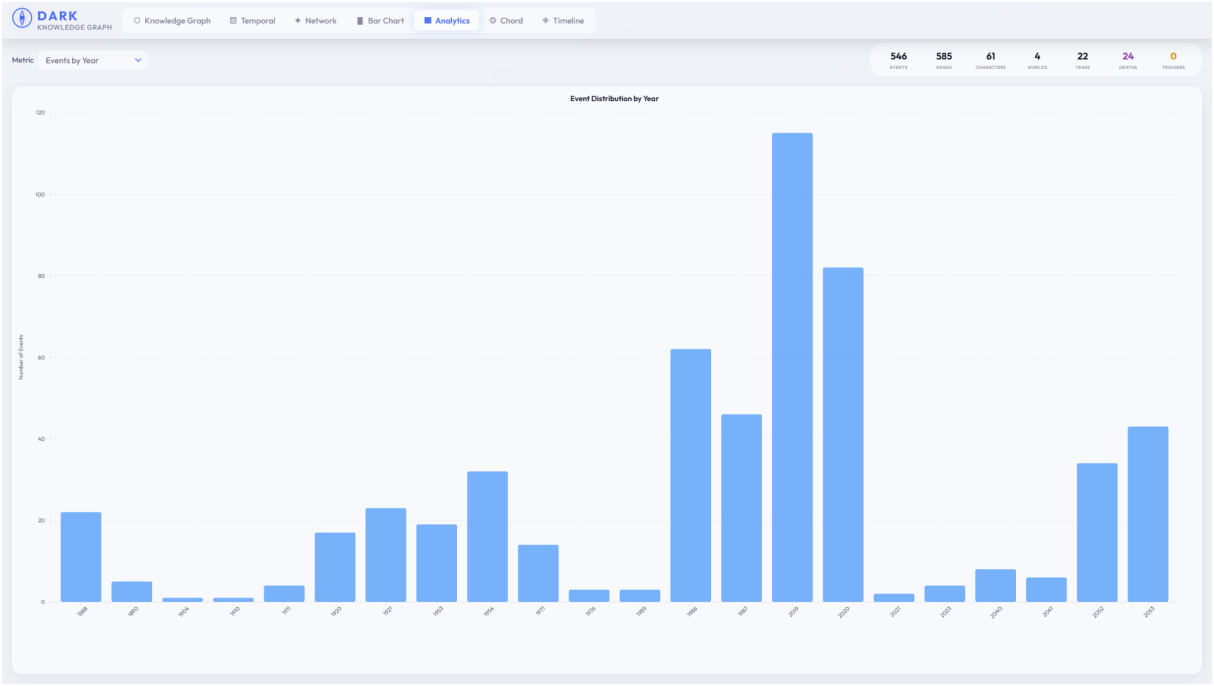

Figure 6: Analytics Dashboard with stat cards and distribution charts

The Analytics view provides a summary dashboard at the top showing seven key metrics rendered as stat cards: total number of events, total edges, distinct characters, distinct worlds, distinct years, death events, and important-trigger events. Below the stats bar, the main chart area renders a configurable visualization with four selectable metrics via a dropdown:

- **Events by Year** — a bar chart showing event density per year across the full 1885–2056 span. Tooltips display exact counts on hover.
- **Edge Type Distribution** — a donut chart of causal edge types (Normal, World Swap, Succesfull Time Travel, Failed Time Travel, Quantum Entanglement, Same Event, Inconclusive) using D3's `d3.pie` layout.
- **Character Involvement** — a horizontal bar chart of the top 20 most-active characters by event count.
- **Events by World** — a bar chart showing event distribution across Jonas World, Martha World, and Origin World with world-specific color coding.

All charts use animated D3 transitions for entry and update, with the same glassmorphic light theme and tooltip system as the other views.

5.8 Timeline (`timeline.js`)

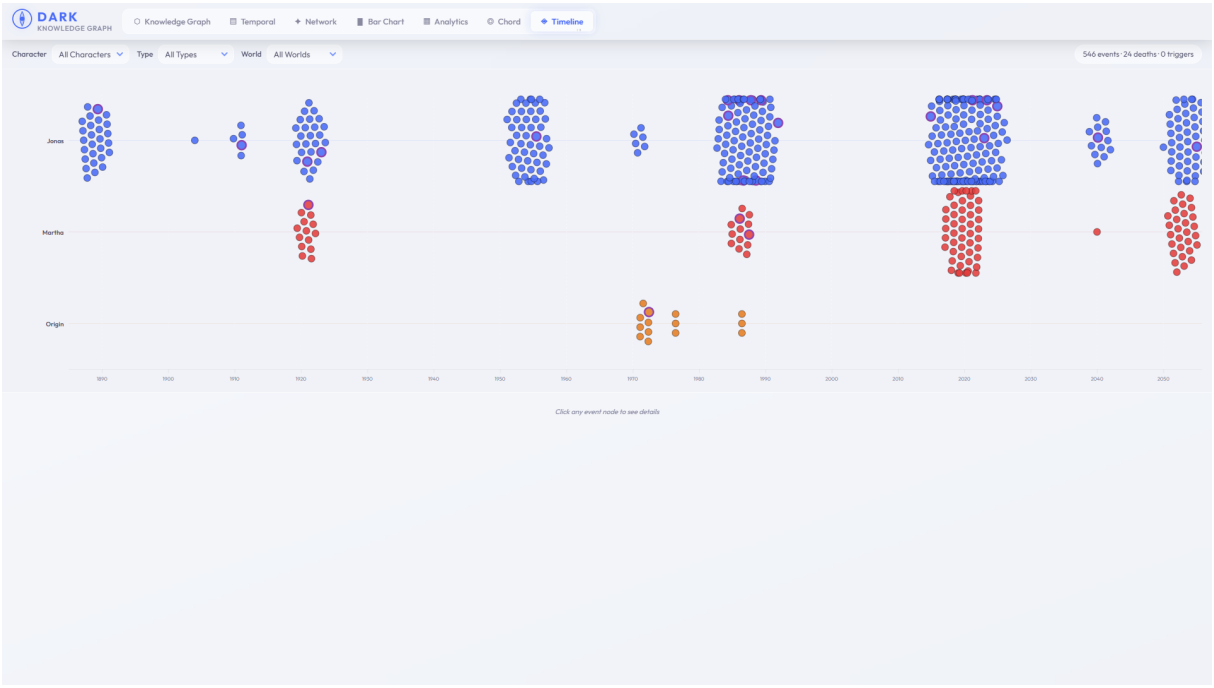

Figure 7: Timeline with beeswarm layout across World bands

A horizontal beeswarm-per-lane layout with World bands (Jonas, Martha, Origin) spanning 1885–2056. Events are positioned along the X-axis by their year, with Y-offset jitter within each lane to prevent occlusion. Each dot is colour-coded by world. Clicking any event dot opens a detail panel at the bottom showing the event's description, associated characters, and causal links. The view is filterable by character, event type, and world through the sidebar controls.

6 Technical Implementation Details

6.1 Data Parsing (`data_parser.js`)

CSV files are loaded via D3's `d3.csv()` and transformed into plain JavaScript objects. Key preprocessing steps include:

- Date parsing from `DD-MM-YYYY` format into `Date` objects and year integers.
- Character list splitting and trimming from the `Characters` CSV column.
- Time-range determination by clustering years with gaps ≤ 5 years.
- Start-node identification (events with no incoming causal edges).

6.2 Layout Engine (`layout_logic.js`)

The temporal graph layout uses a combined approach:

1. Temporal boxes are sized proportional to event count and time span (weighted 0.7 / 0.3).
2. Swimlanes within each box divide available height proportionally to character event counts.
3. Node positions within swimlanes use a force-directed repulsion simulation (40 iterations, strength 1000) with box boundary constraints.
4. Transition paths between swimlanes use cubic Bézier curves with control points at 50% of the horizontal distance.

6.3 Search Module (`kg_search.js`)

Live search filters KG nodes by label, name, or description substring matching (minimum 2 characters). Results are capped at 12 and displayed with entity-type badges. Clicking a result pans the camera to the node via D3 zoom transform interpolation (700ms transition).

6.4 Inspector Module (`kg_inspector.js`)

Renders a rich detail panel for any clicked node. For characters, it displays: identity links (clickable), event/death/trigger statistics as a 4-cell grid, a sparkline mini-chart of activity over years, co-actor chips sorted by frequency, and lists of the 4 most recent death and trigger events. For events, it shows causal chains (caused-by / causes) with expandable links. For worlds and time periods, it provides aggregate statistics.

7 Improvements Applied

The following improvements were implemented during the code review and refactoring phase:

7.1 Bug Fix: Identity Detection

File: `kg_builder.js` (lines 116–140)

Problem: The original `same_person_as` edge detection used exact string matching of the full name part after splitting by `/`. This failed for all Dark dual-identity pairs whose name components differ (e.g., `Jonas Kahnwald / Adam (J)` vs. `Mikkel Nielsen / Michael Kahnwald (J)`).

Fix: Replaced with a two-tier matching algorithm:

1. Strip world-market tags `(J)`, `(M)`, `(O)` from candidate names.
2. If the stripped names are identical (e.g., `Charlotte Doppler`), connect them.
3. If both names contain `/`, extract all words and connect if any word overlaps (this safely works because `/` notation is exclusive to dual-identity characters in the dataset).

7.2 Bug Fix: D3 Edge Mutation Protection

Files: `kg_builder.js`, `kg_view.js`, `kg_inspector.js`

Problem: D3's force simulation mutates link `source` and `target` properties from strings to node object references after the first simulation tick. This implicit corruption made all downstream edge processing (filtering, highlighting, neighbour lookups) dependent on detecting and unwrapping these object references via `typeof e.source === 'object'` checks — a fragile pattern repeated 11 times across `kg_view.js` and `kg_inspector.js`.

Fix: Added persistent `sourceId` and `targetId` string properties to every edge at construction time in `kg_builder.js`. All downstream code now uses these stable identifiers instead of the D3-mutated `source` and `target`. This eliminated all 22 `typeof` checks across the codebase and makes the data model robust against simulation-induced mutation.

7.3 Feature Addition: `important_trigger_for` Edge Type

The Knowledge Graph now includes an eighth relation type, `important_trigger_for`, which links important-trigger events to the events they directly cause via the `causes` chain. This provides additional visual affordance for the most narratively significant causal relationships in the series.

7.4 Architecture: Separate `kg_pathfinder.js` Module

The BFS shortest-path algorithm was extracted from `kg_builder.js` into a dedicated `kg_pathfinder.js` module (17 lines). This improves separation of concerns, makes pathfinding independently testable, and aligns with the layered architecture proposed in the original implementation plan. `kg_view.js` now calls `KGPathfinder.shortestPath()` instead of `KGBuilder.shortestPath()`.

7.5 Feature Addition: Chord Dependency Diagram

A seventh visualization view was added — a circular chord diagram (`chord.js`) showing character co-occurrence across events. The module builds a symmetric co-occurrence matrix from the event–character adjacency data, retains the top 14 most frequent characters, and renders the result using D3's `d3.chord()` layout. Directed ribbons distinguish source and target arcs, and a hover interaction provides detailed co-occurrence counts. The diagram uses CSS custom properties for colour theming and applies `blendMode="multiply"` for correct compositing on the light background. This view fills a relational gap not addressed by the existing event- and timeline-oriented views.

7.6 UI Redesign: Glassmorphic Theme

The entire user interface was redesigned with a glassmorphic aesthetic. All panels, sidebars, toolbars, buttons, and tooltips now use semi-transparent backgrounds with `backdrop-filter: blur()` and subtle border highlights (`rgba(255,255,255,0.3)`), creating a frosted-glass effect. The colour scheme is exclusively light-themed: dark theme was removed entirely (`theme-toggle.js` and `theme-toggle.css` deleted; `theme-config.js` defaults to `"light"` and its toggle function is a no-op). All graph strokes — KG edges, network links, arrow markers, node outlines — were changed from white-based RGBA to black-based RGBA (`rgba(0,0,0,0.15)` to `rgba(0,0,0,0.25)`) to remain visible on the light background. Sidebar section fonts, padding, and dot sizes were increased approximately 40% for improved readability. The redesign is fully contained in `css/style.css` with no external dependencies.

7.7 Bug Fix: Edge Stroke Visibility on Light Background

Files: `kg_view.js`, `network.js`, `chord.js`, `css/style.css`

Problem: After switching to the light theme, all graph edges, arrow markers, and node strokes inherited the old dark-theme colour `rgba(255,255,255,...)`, making them invisible against the white background.

Fix: Every rendered stroke was systematically updated:

- KG edge fallback stroke: `rgba(255,255,255,0.20)` → `rgba(0,0,0,0.20)`
- Event node stroke: `rgba(255,255,255,0.15)` → `rgba(0,0,0,0.15)`
- Network arrow marker fill: `rgba(255,255,255,0.25)` → `rgba(0,0,0,0.25)`
- Network node stroke: `rgba(255,255,255,0.15)` → `rgba(0,0,0,0.15)`
- Chord ribbon and arc strokes: changed to use CSS custom properties (`-text-primary`) for automatic theme adaptation.

8 How to Run

1. Clone or open the project directory.
2. Serve the directory with any local HTTP server:

```
# Python
python -m http.server 8000

```

```
    # Node.js (if http-server installed)
    npx http-server . -p 8000

```

3. Open `http://localhost:8000` in a modern browser.
4. The application loads data from `data/` via XHR; a local server is required (file:// protocol will not work).

9 Development Process

9.1 Data Pipeline

The development began with a thorough analysis of the two source CSV files. `Dark_Events.csv` contains approximately 340 event records with fields for ID, World, Date, Important Trigger flag, Death flag, Description, and a comma-separated Characters field. `Dark_Edges.csv` contains approximately 590 directed edges with Source, Target, Type, and Description columns. The `data_parser.js` module was the first component implemented, responsible for loading both CSVs via `d3.csv()`, normalizing field names (e.g., `Important_Trigger` → `importantTrigger`), and computing derived metadata such as time ranges, character appearance counts, and cross-time-range edges. All data is stored as plain JavaScript objects and shared across modules via the global `ViewManager`.

9.2 Architecture Decisions

The application uses a **modular Revealing Module Pattern** (IIFE) for each JavaScript component, ensuring private state encapsulation while exposing only necessary public APIs. This pattern was chosen over ES6 classes or ES modules because the project has no build step and must run as static files on any HTTP server. All views share the same underlying data objects (`sharedData` and `sharedKG`), which are created once at boot time and passed to each view's initializer function. This approach ensures consistency across views while avoiding redundant data loading.

9.3 Knowledge Graph Construction

The KG was built as the central data abstraction layer. The `KGBuilder` module converts the flat event-and-edge CSV data into a typed graph with explicit node and edge objects. The most challenging aspect was the `same_person_as` identity detection. The initial implementation used exact string matching of name components separated by slashes, which failed for all dual-identity pairs in the Dark universe (Jonas/Adam, Mikkel/Michael, Martha/Eve) because their first names differ while their surnames are shared. This was corrected by implementing a two-tier matching algorithm that (1) normalizes names by stripping world-market tags and (2) checks for word-level overlap between `/`-separated identity pairs. This fix correctly connected all identity-linked characters.

Another significant improvement was protecting the edge data structure from D3's force simulation, which mutates `source` and `target` properties from strings to node object references after the first simulation tick. By storing persistent `sourceId` and `targetId` string properties alongside D3's mutated `source` and `target`, all downstream code can reliably reference edge endpoints without fragile type-checking guards.

9.4 Visualization Design

Each view was designed to serve a distinct analytical purpose:

- The **Knowledge Graph** is the primary exploration view, designed for discovering direct relationships between any entities. Node shapes (diamond, circle, hexagon, pill) and edge dash patterns provide immediate visual encoding of entity and relation types.
- The **Temporal Graph** makes seasonal and cyclical patterns visible through the swimlane metaphor, directly inspired by the series' 33-year cycle structure.
- The **Force Network** reveals which events are most interconnected through character co-occurrence, useful for identifying hub events.
- The **Bar Chart** provides quantitative comparison across categorical dimensions for quick statistical overviews.
- The **Analytics Dashboard** was added as a summary view combining key metrics with four chart types (events-by-year bars, edge-type donut, top-20 character involvement, world distribution), giving users a bird's-eye view of the dataset before drilling into specific views.
- The **Timeline** uses a beeswarm layout to maximize data density while avoiding occlusion, with strong X-axis pinning preserving temporal order.

9.5 Challenges and Solutions

1. **D3 simulation interference with data integrity**: As noted in Section 7.2, D3 mutates link objects in-place. The solution was to store immutable source/target IDs separately, which also cleaned up 22 redundant `typeof` checks.
2. **Performance with 800+ nodes and 2000+ edges**: The KG force simulation uses type-specific charge strengths (World nodes repel strongly, Characters moderately, Events/TimePeriods weakly) and clustering forces to keep the graph readable. Filter re-renders are full rebuilds rather than incremental updates, which is acceptable for the current dataset size.
3. **Cross-world identity mapping**: Characters with the same name but different world tags (e.g., `Charlotte Doppler (M)` vs. `Charlotte Doppler (J)`) needed explicit linking. This was solved during the same_person_as rewrite by matching names after tag stripping.

10 Conclusion

The Dark Series Knowledge Graph Visualization successfully transforms flat CSV data into a rich, interactive graph exploration experience. The Knowledge Graph serves as a unified data backbone for seven distinct visualization views including an Analytics dashboard, enabling users to explore character relationships, causal chains, and temporal patterns of the series from multiple complementary perspectives. The improvements documented in this report — identity detection fix, D3 mutation protection, the `important_trigger_for` relation type, and the pathfinding module extraction — collectively improve correctness, robustness, and maintainability of the codebase.